

Mocap to matched camera images

A simplified explanation of the complete process

The task is to make a reconstructed room appear from the same camera viewpoint as a recorded Crazyflie image, then give the render a comparable camera response. The room was captured at different times, so some objects and lighting differ.

Real camera image

3DGS at the fitted viewpoint



One recorded frame and its simulation at the fitted viewpoint. Geometry and brightness are separate parts of the problem. Real pixels are preserved.

The order matters

Synchronize time

Pair images with mocap.

→ Find shared room points

Associate real and rendered features.

→ Correct every pose

Search camera rotation and position.

Then render the corrected poses, fit the simulated brightness/detail, and inspect the images, overlays and separate error measures.

Start with the correct time and camera pose

The camera clock and the motion-capture clock do not necessarily agree. Fit the clock drift and offset, then test a remaining delay against the motion visible between real images:

$$t_q = t_{h,0} + a(t_d - t_{d,0}) + b + \Delta.$$

t_d is camera-device time; $t_{h,0}$ is the first host timestamp; a, b map the centered device clock to host time; Δ is the remaining image-to-mocap delay. t_q is the time at which to query mocap. All terms use the same time unit.

Interpolate body position and orientation to t_q , then apply the calibrated camera mount: $T_{MC} = T_{MB}T_{BC}$. Here T_{AB} maps coordinates from frame B into frame A : M is mocap, B is the tracked body and C is the camera. Missing tracking or large jumps are recorded as invalid poses.

A recreated Motive rigid body can change its axes and pivot. Verify the camera-to-body transform whenever the tracked body or mount changes.

Find the same physical features

Use the same camera geometry

Render with the Crazyflie’s lens model so the simulated image has the same raw pixel geometry as the recorded image. The real image is left unchanged. For geometric calculations, invert the lens only at the measured feature points:

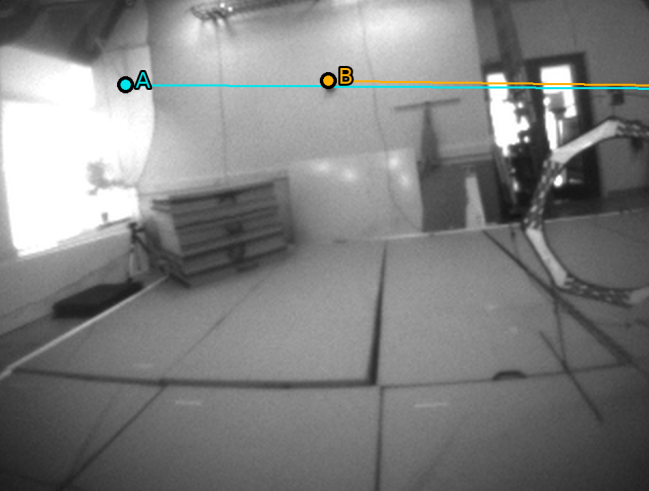
$$p = \begin{bmatrix} (u - c_{0x})/f_x \\ (v - c_{0y})/f_y \end{bmatrix}.$$

(u, v) is the point after lens inversion; f_x, f_y are focal lengths in pixels; (c_{0x}, c_{0y}) is the camera principal point. p is its normalized camera ray coordinate. The active lens is an empirical calibration for the configured camera.

Bootstrap a usable starting alignment

Render several known viewpoints of the room. Match real images to those views, lift matched rendered pixels using scene depth, and estimate provisional camera poses. These visual poses initialize the room transform and camera-axis correction. Recheck timing after applying the camera correction.

Real image



Render used for matching



Matching letters/colors identify the same stored candidate association on both sides. Only a few candidates are drawn for clarity. These stored candidates illustrate the visual review step.

Turn repeated observations into a fixed scene point

SuperPoint detects image locations; LightGlue, with MINIMA weights for the real/render pair, associates them. Match across rendered viewpoints, then intersect the rays from their known camera poses:

$$X_j = \arg \min_X \sum_k \left\| (I - d_k d_k^T)(X - p_k) \right\|^2.$$

X is a candidate point and X_j the recovered room point; p_k is the rendering camera centre, d_k the unit viewing ray and I the identity matrix. The expression measures how far a candidate point lies from all its viewing rays.

Keep points that agree across views, have useful viewing-angle separation and reproject consistently in real images. Review the associations visually and reject changed objects or repeated-pattern mistakes. Tracking propagates an association through nearby frames only when consistency checks pass.

Feature identity means the same physical location, not merely a similar-looking corner. Floor, gate and other scene-specific exclusions keep the room fit focused on stable background structure.

Search the camera pose frame by frame

First: one global initialization

Use observations of admitted room points to establish the common scene registration. The initial cameras receive a shared rotation, translation and scale correction:

$$R'_i = R_c R_i, \quad p'_i = e^s R_c (p_i - c) + c + \delta, \quad R_c = \exp([\omega]_{\times}).$$

R_i, p_i are the starting camera orientation and position; ω describes the correction rotation; δ is its translation; e^s is the distance multiplier. $[\omega]_{\times} v = \omega \times v$ defines the rotation's skew matrix. c is the mean starting camera centre, used as a rotation/scale pivot.

Then: an independent correction for each observed frame

Starting from the global fit, each frame has its own rotation ω_i and translation δ_i :

$$R'_i = \exp([\omega_i]_{\times}) R_i, \quad p'_i = p_i + \delta_i.$$

Project every known room point X_j into the candidate camera pose and compare its predicted position $\hat{u}_{ij}$ with the observed feature u_{ij} . Both are in the same lens-corrected pixel coordinates.

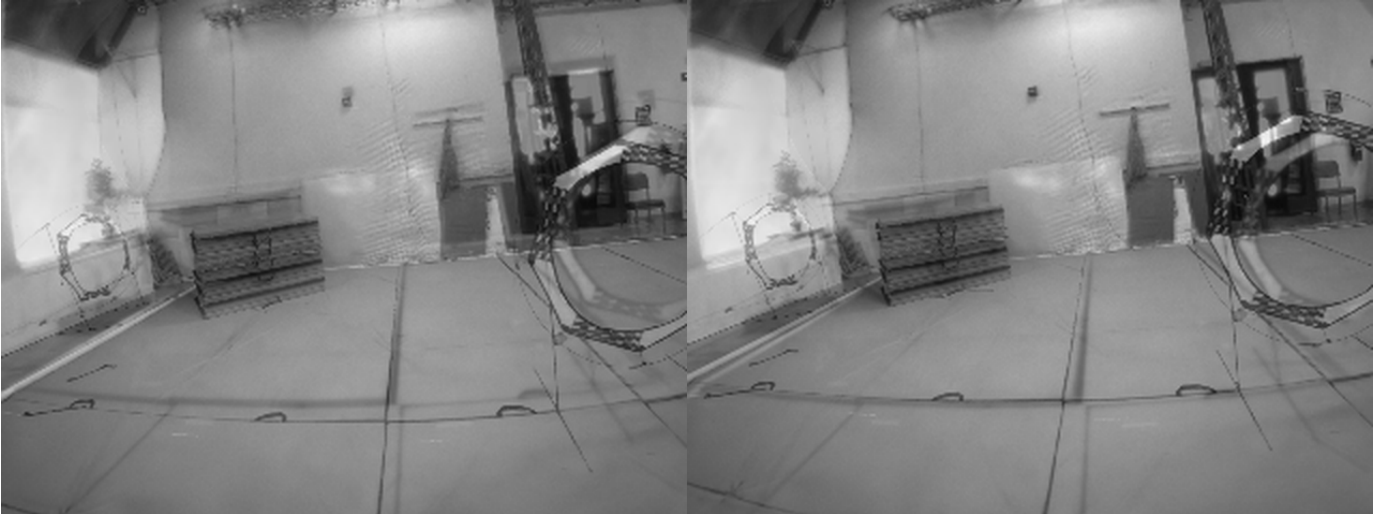
$$L(\theta) = \frac{1}{W} \sum_j w_j \rho\left(\frac{\|\hat{u}_{ij} - u_{ij}\|}{\sigma_j}\right) + \frac{1}{2W} \sum_k \left(\frac{\theta_k}{\tau_k}\right)^2.$$

θ contains the candidate pose correction. w_j is an observation's weight, $W = \sum_j w_j$, and σ_j is its pixel uncertainty. τ_k sets a weak allowance for each correction component. The Huber penalty $\rho(e)$ is $e^2/2$ for $e \leq 1$, and $e - 1/2$ otherwise: large feature errors have less influence.

Search loop: evaluate a coarse grid of candidate poses; retain the best candidates; search smaller grids around them; finish with local refinement of the same loss. Repeat this process using each frame's own observations.

Overlay at the starting pose

Overlay after per-frame pose fitting

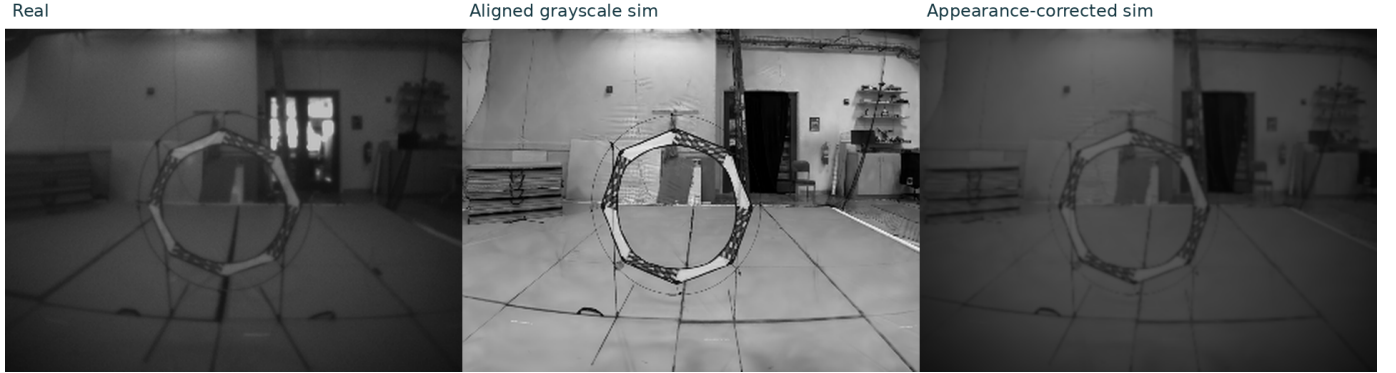


Real image blended with a starting render and a per-frame render of the same frame. Doubled room edges help reveal a remaining pose mismatch. Use fixed room edges to assess the pose independently of brightness.

The global fit is only the starting point. Frames with no usable observations can receive a nearby/interpolated correction; longer unsupported gaps retain the global pose. The dataset labels these cases separately from independently fitted frames.

Make the simulated appearance comparable

Freeze the fitted poses before changing appearance. Convert the RGB render to grayscale, then fit a shared response to the real recordings. This separates camera/lighting differences from geometric misalignment.



An example from a darker recording. The middle panel changes geometry only; the right panel applies the frozen appearance model.



An example looking toward bright light. Saturation and missing reconstructed detail remain visible; the response cannot recover information that was not captured.

$$\hat{I}(p) = \text{clip}_{[0,1]} \left[b + a(eh)^\beta \bar{g}(p)^\gamma \exp\{-v_1 q(p) - v_2 q(p)^2 + D(p)\} \right].$$

$\bar{g}(p)$	Grayscale simulated intensity at pixel p , with a small selected blur.
a, b, γ	Shared brightness gain, black offset and tone-curve exponent.
e, h, β	Relative exposure times digital gain, a known-run correction, and the fitted exposure-response exponent.
$q(p), v_1, v_2$	Squared image radius and the radial brightness coefficients.
$D(p)$	Directional response from the scene-facing ray, plus a frame-level term from viewing direction and simulated bright content.

Fit the bounded coefficients using shared-feature pixels, uniformly distributed tile averages, radial brightness and dark-image statistics. Select blur and model form on development frames, then freeze the choice before checking reserved frames. Tile averages balance where the loss is measured while one shared response corrects the image.

At inference, the correction requires the render, its camera orientation and recording settings. The empirical response approximates the combined appearance of scene lighting and the camera.

Measure image and feature agreement

Keep geometric error and image error separate

For feature reprojection distances $e_j = \|\hat{u}_j - u_j\|$, report

$$\text{RMS}_{\text{feature}} = \sqrt{\frac{1}{N} \sum_j e_j^2}, \quad \text{median}_{\text{feature}} = \text{median}_j e_j.$$

N is the number of compared observations. RMS emphasizes large misses; the median describes a typical observation. Both measure feature agreement in image coordinates.

For image residuals $d_p = \hat{I}(p) - I(p)$ over the fixed comparison region Ω , use

$$\text{MAE} = \frac{1}{|\Omega|} \sum_{p \in \Omega} |d_p|, \quad \text{RMSE} = \sqrt{\frac{1}{|\Omega|} \sum_{p \in \Omega} d_p^2}.$$

I is the real image and $\hat{I}$ is the simulated image, both on the same intensity scale. RMSE is the square root of mean squared error. Signed mean residual reveals whether the render is systematically brighter or darker.

Measure structure and edge agreement

SSIM compares local brightness, contrast and image structure.

Edge precision/recall measure whether rendered and real edges occur near each other. Combine them as $F_1 = 2PR/(P + R)$, where P is precision and R is recall.

Use the separate components in a predeclared selection objective that also penalizes radial brightness bias. This discourages a very dark or blurred render from winning solely through lower pixel error. Report full-image and stable-feature-region comparisons separately.



The review product: real image, appearance-corrected simulation and their overlay. Look at fixed edges, black level, contrast and highlights independently.

Read the outputs

The pipeline saves synchronized pose validity, reviewed feature associations, per-frame poses and fit modes, raw-geometry renders, corrected grayscale images, and videos with an explicit frame/time mapping. These allow every step to be checked against the original footage.

Moved objects, reconstruction gaps, glare, saturation and calibration error can remain. Timing, lens, camera mount and feature identity need independent checks; an appearance fit cannot repair them. Reserved images from the same recordings test appearance conditional on the fitted poses, not transfer to a new room or an independently calibrated camera.

This guide explains the method with recorded examples. Reproduction commands, coefficient files and the full mathematical details are in the repository README and method guide.